THM | CAMPUS FRIEDBERG | IEM
TECHNISCHE HOCHSCHULE MITTELHESSEN
Informationstechnik-Elektrotechnik-Mechatronik

Prozessorhardware mit VHDL

Interrupt-Controller

für die Plasma-CPU

Philipp Tofahrn

30. September 2017

Technische Hochschule Mittelhessen
Prof. Weber

Inhaltsverzeichnis

1. Einleitung **3**

2. Problemstellung **4**

3. Umfang des Projektes **6**

3.1. Anpassen des Interrupt-Vektors 6
3.2. Interrupt-Controller 6
3.3. Überarbeiten der Interrupts 7
3.4. Beispielanwendung 7

4. Umsetzung **8**

4.1. Vektortabelle 8
4.2. Toolchain & Boot-Code 9
4.3. Interrupt-Controller 9
4.4. Interrupt-Handling 10
4.5. Erweitern des Zählers 11
4.6. GPIO-Interrupts 11
4.7. Konfiguration der Interrupts 12
4.8. Beispielanwendung 12

5. Ausblick **14**

A. Tabellen- und Abbildungsverzeichnis **15**

1. Einleitung

Im Rahmen des Moduls „Prozessorhardware mit VHDL“ wurden die Grundlagen der Prozessorentwicklung besprochen. Dabei wurde im weiteren Verlauf speziell auf den MIPS-Prozessor eingegangen. Für den praktischen Teil der Vorlesung wurde die sogenannte „Plasma-CPU“[1] verwendet. Dies ist eine CPU, welche den Befehlssatz der MIPS I Architektur weitestgehend unterstützt.

Im Laufe des Semesters wurde deutlich, dass die Plasma-CPU noch Potential zur Verbesserung hatte. Insbesondere das Interrupt-System, welches ich mir als Abschlussprojekt vorgenommen habe, erwies sich als verbesserungswürdig.

[1] `http://opencores.org/project,plasma`

2. Problemstellung

Die Plasma-CPU hatte bislang nur ein sehr eingeschränktes Interrupt-System. Für dieses konnten verschiedene Quellsignale über eine Maske ausgewählt werden, darüber hinaus waren die Interrupts jedoch nicht konfigurierbar. Die maskierten Signale wurden dann zu dem Interrupt-Request ODER-verknüpft. Dies ist schematisch in Abb. 2.1 dargestellt.

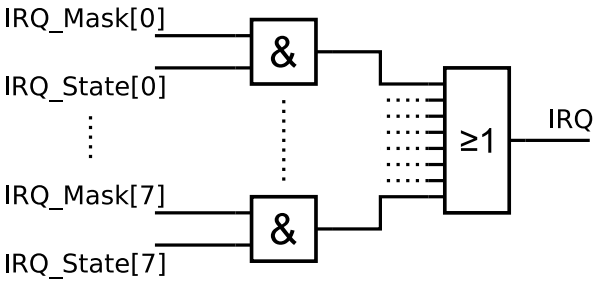


Abbildung 2.1.: Ermittlung des Interrupt-Requests.

Um zu analysieren, welches Signal einen Interrupt ausgelöst hat, musste das Programm die Speicheradresse `0x20000020` (`IRQ_STATUS`) auslesen. Dies gab jedoch keinen zuverlässigen Status an die Software weiter, da die Interrupt-Signale nicht in einem Register gesichert wurden. Dadurch konnte es vorkommen, das das Bit des ausgelösten Interrupts nicht mehr gesetzt war, bis der Interrupt-Handler ausgeführt wurde.

Dem Interrupt-System fehlte insbesondere die Möglichkeit Signalflanken zu detektieren. Auch dies musste die Software lösen. Dazu lagen die möglichen Eingangssignale sowohl in negierter als auch nicht-negierter Form vor. Der Interrupt-Handler musste dann die Maske entsprechend bearbeiten, damit die CPU nicht im Interrupt-Status gehalten wurde.

Auch die zur Verfügung stehende Auswahl an Signalen schränkten die Nutzbarkeit sehr ein. Beispielsweise gab es für Timer-Interrupts nur die Möglichkeit, einen Interrupt auf Bit 18 des Zählers zu setzen. Eine Übersicht der Signale ist in Abb. 2.2 dargestellt.

7	6	5	4	3	2	1	0
gpioA[31]	gpioA[30]	$\overline{\text{gpioA[31]}}$	$\overline{\text{gpioA[30]}}$	cnt[18]	$\overline{\text{cnt[18]}}$	$\overline{\text{uart busy}}$	uart data

Abbildung 2.2.: Aufbau von `IRQ_MASK` und `IRQ_STATUS`.

Bislang sprang die Plasma-CPU bei einem Reset an Adresse 0 und bei einem Interrupt an `0x3C`. Dies war dadurch begründet, dass der Startup-Code, welcher an Adresse 0 liegt, exakt aus 15 Instruktionen bestand. Dadurch wurden Änderungen am Boot-Code sehr erschwert, da der VHDL-Code angepasst werden musste.

3. Umfang des Projektes

Ausgehend von der Problemstellung wurden im Rahmen dieses Projektes die folgenden Lösungsansätze erarbeitet:

3.1. Anpassen des Interrupt-Vektors

Um die Arbeit mit der Plasma-CPU in Zukunft zu vereinfachen, sollte der Interrupt-Vektor so überarbeitet werden, dass er nicht mehr von einer spezifischen Anzahl an Instruktionen im Boot-Code abhängig ist. Dazu gehört auch eine Anpassung der Toolchain, um diese Änderung zu unterstützen.

3.2. Interrupt-Controller

Die Plasma-CPU sollte um einen Interrupt-Controller erweitert werden, welcher die gängigen Konfigurationsmöglichkeiten unterstützt. Diese sind im Speziellen:

- Level-Interrupts
- Flanken-Interrupts
- wählbare Polarität

Außerdem sollte die Software zu jeder Zeit herausfinden können, welches Quellsignal für das Auslösen des Interrupts verantwortlich ist.

3.3. Überarbeiten der Interrupts

Da durch die Konfigurierbarkeit der Interrupts die vorhandenen Signale teilweise unnötig wurden und andere Signale zu eingeschränkt nutzbar waren, mussten auch diese überarbeitet werden.

Der Zähler sollte flexibler einsetzbar werden. Um dies umzusetzen musste der Interrupt-Intervall konfigurierbar sein. Dadurch kann eine Applikation ein Zeitintervall wählen, welches für sie nützlich ist.

Bisher mussten die GPIO-Signale in negierter und nicht-negierter Form als Quellsignale vorliegen. Diese Dopplung wird von dem neuen Interrupt-Controller nicht mehr benötigt. Hier galt es also, eine flexiblere Variante zu entwickeln.

3.4. Beispielanwendung

Um die Überarbeitung der Plasma-CPU zu testen, sollte eine Beispielanwendung geschrieben werden. Ziel dieser war es, die neuen Features zu nutzen und als abschließende Kontrolle für die erfolgreiche Umsetzung zu dienen.

4. Umsetzung

4.1. Vektortabelle

Die Plasma-CPU wurde so überarbeitet, dass der interne Speicher mit einer Interrupt-Vektortabelle[1] beginnt. Jeder Eintrag besteht aus vier Speicherworten, um Platz für einen Longjump zu bieten:

```
lui $k1 ...
ori $k1 ...
jr  $k1
nop
```

Hierfür wird eins der beiden Kernel-Register (`$k1`) verwendet, welches speziell für diesen Zweck reserviert wird.

Bislang gibt es genau zwei Einträge in der Vektortabelle, diese sind in Tabelle 4.1 dargestellt.

Adresse	**Vektor**
0x00	Reset-Vektor
0x10	Interrupt-Vektor

Tabelle 4.1.: Aufbau der Interrupt-Vektor-Tabelle.

[1] `https://en.wikipedia.org/wiki/Interrupt_vector_table`

4.2. Toolchain & Boot-Code

Für die volle Unterstützung der neuen Vektortabelle musste die Toolchain angepasst werden. Hier wurde der Weg gewählt, den Linker per Linkerskript zu konfigurieren. Dies brachte einige Vorteile mit sich:

- neues Segment für die Vektortabelle
- sichergestellte Adressen für die Einträge der Vektortabelle
- Generierung von Symbolen für den Stack-Pointer, Global-Pointer, etc.

Durch diesen Ansatz konnten zusätzlich einige Aufgaben des Convertes in den Linker verlagert werden. Der alte Converter musste einige Instruktionen modifizieren, um den Stack-Pointer und Global-Pointer zu initialisieren. Der neue Converter übernimmt nur noch die Umwandlung von einer `ELF` in `vhdl`- und `bin`-Dateien.

4.3. Interrupt-Controller

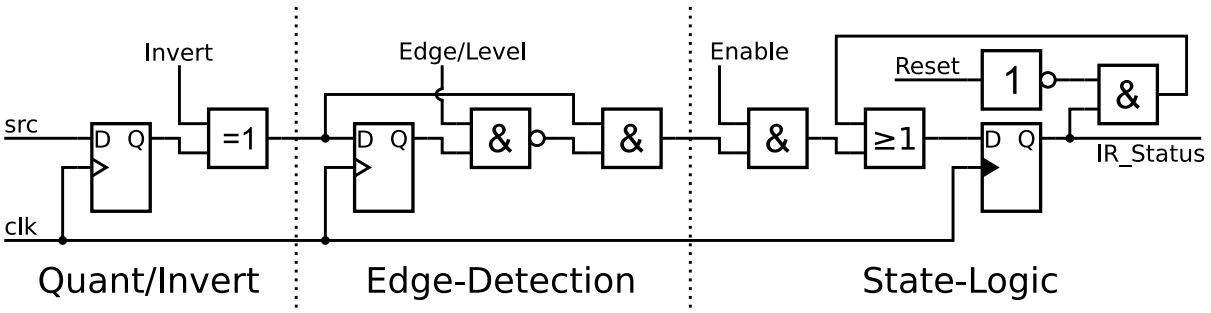


Abbildung 4.1.: Verarbeitung eines Interrupt-Signals.

Die Verarbeitung eines Interrupts lässt sich in 3 Stufen unterteilen. In Stufe 1 wird das Eingangssignal quantisiert. Dies ist für die in Stufe 2 folgende Flankenerkennung zwingend notwendig. Außerdem erfolgt hier gegebenenfalls die Invertierung des Eingangssignals.

In Stufe 2 erfolgt die Flankenerkennung, falls diese aktiviert ist (Edge/Level = 1). In diesem Fall erzeugt eine positive Flanke einen Impuls von einem Taktzyklus.

Stufe 3 beinhaltet das Speichern des Interrupt-Status. Das Interrupt-Flag wird so lange aktiv gehalten, bis die Software ein Reset-Signal erzeugt. Das Set-Signal hat dabei allerdings immer Vorrang, sodass ein noch aktiver Level-Interrupt sich nicht zurücksetzen lässt.

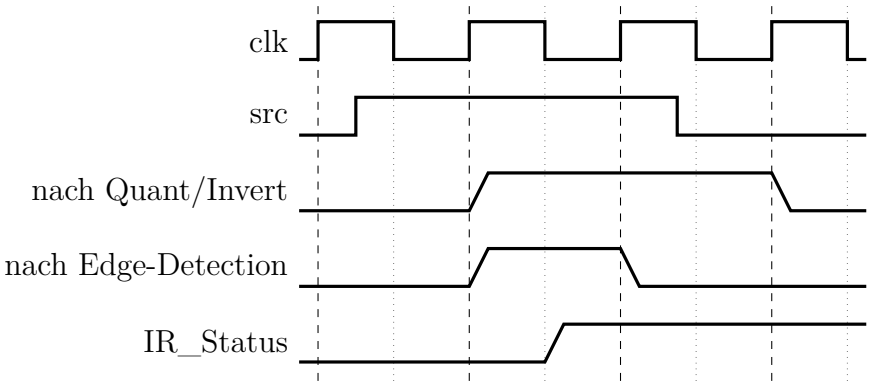


Abbildung 4.2.: Beispielhaftes Timing-Diagramm der Interrupt-Logik in Abb. 4.1. (Konfiguaration: Edge/Level = 1; Invert = 0; Enable = 1).

Wie in Abb. 4.2 zu sehen, beträgt die Verarbeitungszeit eines Interrupts einen Taktzyklus. Das bedeutet, dass die Auslösung des Interrupt-Status der CPU im *Worst Case* zwei Taktzyklen beträgt. Dies ist der Fall, wenn das Quellsignal direkt nach der steigenden Flanke des Clock-Signals seinen Status ändert.

Nachteil dieser Technik ist, dass Quellsignale, welche kürzer als einen Taktzyklus andauern, nicht sicher einen Interrupt auslösen.

4.4. Interrupt-Handling

Der in Assembler geschriebene Interrupt-Handler ist dafür zuständig, die Register zu sichern und dann den entsprechenden C-Callback aufzurufen. Dazu muss dieser die gesetzten Interrupt-Flags auszuwerten. Nach Rückkehr des Aufrufs setzt der Handler das Interrupt-Flag zurück. Der Ablauf ist in Abb. 4.3 dargestellt.

Um festzustellen, welches Quellsignal den Interrupt ausgelöst hat, ist es nötig zu ermitteln, welches Bit des Interrupt-Status gesetzt ist. Für den Interrupt-Handler wurde die einfachste Art der Ermittlung über eine Schleife gewählt.

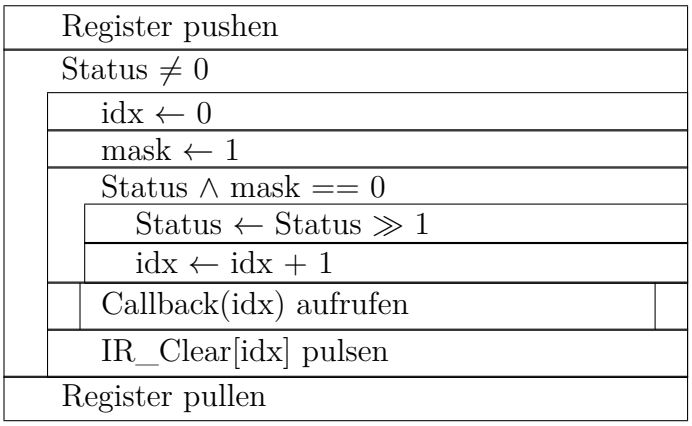


Abbildung 4.3.: Ablaufdiagramm des Interrupt-Handlers.

Die Instruktionen zum Sichern aller Register und zur Ermittlung des ausgelösten Interrupts benötigen im besten Fall 35 Taktzyklen. Der beste Fall ist hierbei, wenn der Interrupt mit der höchsten Priorität (Bit 0 des Interrupt-Status) ausgelöst hat. Für jedes nicht gesetzte Bit werden weitere 4 Zyklen benötigt.

4.5. Erweitern des Zählers

Die Arbeitsweise des Zählers wurde komplett geändert. Statt eines Aufwärtszählers wird jetzt ein Abwärtszähler verwendet. Beim Erreichen des Wertes `0` wird der *Counter Interrupt* ausgelöst. Im nächsten Zyklus wird der Zähler dann mit dem *Counter Reload Value* neu geladen.

4.6. GPIO-Interrupts

Für möglichst flexible GPIO-Interrupts wurden diese konfigurierbar ausgeführt. Insgesamt stehen vier dieser Interrupts zur Verfügung. Für jeden kann der GPIO-Pin ausgewählt werden, welches den Interrupt auslösen soll. Der Aufbau der Speicherstelle, an der diese Konfiguration durchgeführt werden kann, ist Abb. 4.4 zu entnehmen. Zusammen mit den Einstellmöglichkeiten des Interrupt-Controllers ergeben sich dadurch hoch flexible GPIO-Interrupts.

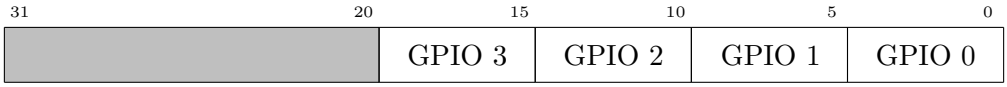


Abbildung 4.4.: Aufbau des GPIO Interrupt Select.

4.7. Konfiguration der Interrupts

Zur Konfiguration der Interrupts wurde der Weg über ein Memory-Mapping gewählt. Die bisherige Konfiguration der Peripherie erfolgte auch über diesen Weg, weshalb es nahe lag, ihn auch hier zu übernehmen.

Die neu belegten Speicheradressen können Tabelle 4.2 entnommen werden.

Adresse		**Beschreibung**
`0x20000080`	r	IR Status Raw
`0x20000090`	r	IR Status
`0x200000A0`	rw	IR Invert
`0x200000B0`	rw	IR Edge
`0x200000C0`	rw	IR Enable
`0x200000D0`	w	IR Clear
`0x200000E0`	rw	GPIO Interrupt Select
`0x200000F0`	rw	Counter Reload Value

Tabelle 4.2.: Memory-Map der neuen Konfiguarationsparameter.

4.8. Beispielanwendung

Um den neuen Interrupt-Controller und die weiteren Modifikationen zu testen, wurde ein Beispielprogramm geschrieben. Dieses konfiguriert zuerst die Interrupts und geht dann in eine Dauerschleife. Sämtliche Funktionen sind als Interrupts realisiert.

Der Button „BTNR“ löst das GPIO-Interrupt 0 aus, welches wiederum bei jedem Aufruf den Zustand der LED „LD0“ umschaltet.

Der *Counter Reload Value* ist auf 25.000.000 gesetzt. Bei einer Taktfrequenz von 50 MHz ergibt sich dadurch ein Timeout-Intervall von ca. einer halben Sekunde. Der Timeout-Interrupt schaltet bei jedem Aufruf den Zustand der LED „LD3“ um.

Des Weiteren empfängt die Firmware Daten per UART. Diese werden von dem „UART DATA“ Interrupt ausgewertet, verändert und zurückgeschickt. Die gesendeten Daten unterscheiden sich von den empfangenen darin, dass das niederwertigste Bit invertiert wurde ($send = receive \oplus 1$).

5. Ausblick

Der Interrupt-Controller ist so geschrieben, dass er sich per *generic* auf eine beliebige Anzahl an Eingängen erweitern lässt. Durch die Speicherbreite von 32-Bit des MIPS werden aktuell bis zu 32 Interrupt-Quellen unterstützt, von denen 7 belegt sind. Die aktuelle Belegung ist in Abb. 5.1 dargestellt. Dadurch lassen sich leicht weitere Interrupts hinzufügen.

Auch eine Erweiterung auf über 32 Interruptquellen ist denkbar, jedoch müssen dazu weitere Speicherbereiche für die Konfiguration vorgesehen werden. Auch der Interrupt-Handler müsste dann entsprechend angepasst werden.

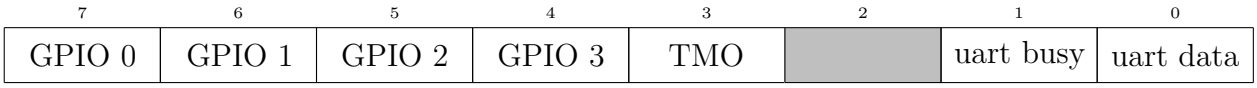


Abbildung 5.1.: Neuer Aufbau von `IRQ_MASK` und `IRQ_STATE`.

Der Assemblercode des Interrupt-Handlers ist nicht auf Performance optimiert, daher existieren hier noch Verbesserungsmöglichkeiten. Des Weiteren wäre die Entwicklung eines Hardware-Prioritätsencoders denkbar. Dieser würde die Aufgabe übernehmen, das niedrigste gesetzte Bit des Interrupt-Status zu ermitteln und dadurch die Software entlasten.

A. Tabellen- und Abbildungsverzeichnis

Abbildungsverzeichnis

2.1. Ermittlung des Interrupt-Requests. 4
2.2. Aufbau von `IRQ_MASK` und `IRQ_STATUS`. 5

4.1. Verarbeitung eines Interrupt-Signals. 9
4.2. Beispielhaftes Timing-Diagramm der Interrupt-Logik 10
4.3. Ablaufdiagramm des Interrupt-Handlers. 11
4.4. Aufbau des GPIO Interrupt Select. 12

5.1. Neuer Aufbau von `IRQ_MASK` und `IRQ_STATE`. 14

Tabellenverzeichnis

4.1. Aufbau der Interrupt-Vektor-Tabelle. 8
4.2. Memory-Map der neuen Konfiguarationsparameter. 12